



Developing Business Rules

Version 2026.1
2026-04-20

Developing Business Rules

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 About Business Rules	1
1.1 Introduction	1
1.2 Rule Definitions	3
1.3 Rule Definitions as Classes	3
1.4 Rule Definitions and Package Mapping	4
1.5 See Also	4
2 Getting Started	5
2.1 About Rule Definitions	5
2.1.1 Exporting and Importing Rules	6
2.2 About Rule Sets	6
2.3 See Also	7
3 Working with Rules	9
3.1 Defining Constraints	9
3.2 About If and Else Clauses	10
3.3 About Actions	10
3.3.1 Using the foreach Action	11
3.4 Disabling a Rule	12
3.5 Passing Data to a Data Transformation	13
3.6 See Also	13
4 Expressions in Business Rules	15
4.1 The context Variable	15
4.2 The Document Variable	15
4.3 Available Operators	15
4.4 Available Functions	17
4.5 Expression Examples	17
4.6 Boolean Expressions	18
5 Debugging Routing Rules	21
5.1 Testing Routing Rules	21
5.1.1 Testing with Raw Text of a Message	21
5.1.2 Test Results	22
5.1.3 Security Requirements	22
5.2 Strategies for Debugging Routing Rules	22
5.3 See Also	27
Appendix A: Utility Functions for Use in Productions	29
A.1 Built-in Functions	29
A.2 Usage Differences between Business Rules and DTL	35
A.3 See Also	35

List of Figures

Figure 5-1: Solving Problems with Routing Rules, Initial Triage — Decision Tree A 23

Figure 5-2: Solving Problems with Routing Rules — Decision Tree B 24

Figure 5-3: Solving Problems with Routing Rules — Decision Tree C 25

Figure 5-4: Solving Problems with Routing Rules — Decision Tree D 26

Figure 5-5: Solving Problems with Routing Rules — Decision Tree E 27

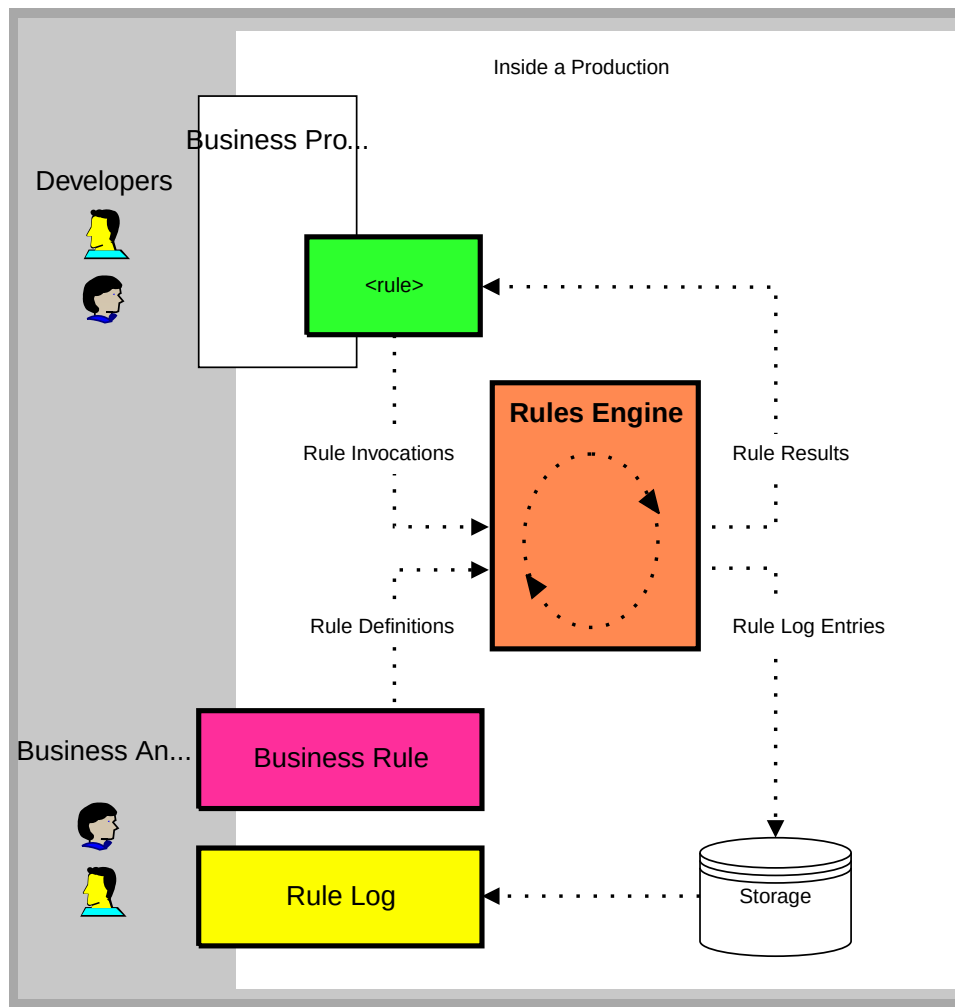
1

About Business Rules

This page introduces *business rules*, which are a form of [business logic](#) you can use within [interoperability productions](#). You can use business rules within a workflow; see [Comparison of Business Logic Tools](#).

1.1 Introduction

Business rules allow nontechnical users to change the behavior of business processes at specific decision points. You can change the logic of the rule instantly, using the Rule Editor in the Management Portal, without any coding. The following figure shows how business rules work.



Suppose that an international enterprise runs a production that processes loan applications. The decision process is consistent worldwide. However, each bank in the enterprise has its own acceptance criteria, which may vary from country to country. Business rules support this division of responsibility as follows:

1. The developer of the business process identifies a decision point, by naming the business rule that will make the decision on behalf of the business process. The developer leaves a placeholder for that business rule in the Business Process Language (BPL) code by invoking the Business Process Language (BPL) element `<rule>`. The `<rule>` element specifies the business rule name, plus parameters to hold the result of the decision and (optionally) the reason for that result. Suppose we call this rule `LoanDecision`.
2. Wherever the `<rule>` element appears in a BPL business process, a corresponding rule definition must exist within the production. A user at the enterprise, typically a business analyst, may define the rule using the browser-based online form called the Rule Editor. This form prompts the user for the simple information required to define the business rule called `LoanDecision`. InterSystems IRIS® saves this information in its configuration database.

Any enterprise user who is familiar with the Rule Editor and who has access to it in the Management Portal can modify the rule definition. Modifications are simply updates to the database and can be instantly applied to a production while it is running. Therefore, it is possible for business analysts at various regional locations to run the Rule Editor to modify their copies of the rule to provide different specific criteria appropriate to their locales.

3. At runtime, upon reaching the BPL `<rule>` statement the business process invokes the rule named `LoanDecision`. The rule retrieves its decision criteria from the configuration database, which may be different at different locales. Based on these criteria, the rule returns an answer to the business process. The business process redirects its execution path based on this answer.

- For ongoing maintenance purposes, the business process developer need not be involved if a rule needs to change. Any rule definition is entirely separate from business process code. Rule definitions are stored in a configuration database as classes and are evaluated at runtime. Additionally, rule definitions can be exported and imported from one InterSystems IRIS installation to another.

In this way, enterprise users such as business analysts can change the operation of the business process at the decision point, without needing the programming expertise that would be required to revise the BPL or class code for the business process.

1.2 Rule Definitions

A rule definition is a collection of one or more rule sets, and a rule set is a collection of one or more rules. Each rule set has an effective (or beginning) date and time as well as an ending date and time. When a business process invokes a rule definition, one and only one rule set is executed.

1.3 Rule Definitions as Classes

Internally all rule definitions are classes, and a developer can use an [IDE](#) to edit them, as an alternative to using the visual rule editor. The following shows an example rule definition class:

Class Definition

```
/// Business rule responsible for mapping an input location
///
Class Demo.ComplexMap.Rule.SemesterBatchRouting Extends Ens.Rule.Definition
{
    Parameter RuleAssistClass = "EnsLib.MsgRouter.RuleAssist";

    XData RuleDefinition [ XMLNamespace = "http://www.intersystems.com/rule" ]
    {
        <ruleDefinition alias="" context="EnsLib.MsgRouter.RoutingEngine"
        production="Demo.ComplexMap.SemesterProduction">
        <ruleSet name="" effectiveBegin="" effectiveEnd="">
        <rule name="" disabled="false">
        <constraint name="source" value="Semester_Data_FileService"></constraint>
        <constraint name="msgClass" value="Demo.ComplexMap.Semester.Batch"></constraint>
        <when condition="1">
        <send transform="" target="Semester_Data_FileOperation"></send>
        <send transform="Demo.ComplexMap.Transform.SemesterBatchToSemesterSummaryBatch"
        target="Semester_Summary_FileOperation"></send>
        <send transform="Demo.ComplexMap.Transform.SemesterBatchToFixedClassBatch"
        target="Semester_FixedClassBatch_FileOperation"></send>
        <send transform="Demo.ComplexMap.Transform.SemesterBatchToFixedStudentBatch"
        target="Semester_FixedStudentBatch_FileOperation"></send>
        <send transform="" target="Semester_FixedStudent_BatchCreator"></send>
        <return></return>
        </when>
        </rule>
        </ruleSet>
        </ruleDefinition>
    }
}
```

Changes made in an IDE might not be immediately visible in the **Rule Editor** page; you may have to refresh the page to see them.

1.4 Rule Definitions and Package Mapping

Given that rules are classes, you can map rules to other namespaces. If you do so, you must recompile all the mapped rule classes in each namespace where you use them to ensure that the local metadata is available in each namespace.

For details, see [Package Mapping](#).

1.5 See Also

- [Getting Started](#)
- [Working with Rules](#)
- [Expressions in Business Rules](#)
- [Debugging Routing Rules](#)

2

Getting Started

This page introduces the Rule Editor in the Management Portal and briefly introduces how to work with [business rules](#) for [interoperability productions](#). Each business rule in InterSystems IRIS is part of a rule set. In turn, each rule set is part of a larger rule definition. Sometimes, the terms *rule* and *rule definition* are used interchangeably, but each individual rule is ultimately grouped under a rule definition.

Rule definitions, rules sets, and rules are edited using the Rule Editor, which is accessed in the Management Portal by navigating to **Interoperability > Build > Business Rules**.

You can open existing rules in the Rule Editor by navigating to **Interoperability > List > Business Rules**.

The Rule Editor has an automatic timeout; you are logged out after 10 minutes of inactivity.

2.1 About Rule Definitions

Individual business rules are grouped under a rule definition, which is built and edited using the Rule Editor. A rule definition includes the following settings:

Package

Package for the rule definition class.

Name

Name of the rule definition class.

Description

User-specified description of the rule definition and its purpose.

Rule Type

Type of rule definition, which determines valid actions when defining rules that belong to the rule definition.

Context Class

Class that determines which object properties you can modify when you edit a rule. For general business rules, the context class is generated from the business process class associated with the BPL process and ends in `.Context`. For routing rules that are not associated with a BPL process, the context class is usually the business process class used by the routing engine.

When creating the rule definition, you can use the **Filters** options to shrink the list of classes that appears in the **Context Class** drop-down list.

Production Name

(Optional) For routing rules, provides the name of the production where the rule will be used so the Rule Editor can offer predefined options when editing the rule. For example, if you specify a production and then modify a constraint, the configuration items in the production appear as options for the **Source** field of the constraint.

The routing rule is not automatically used in the production unless you specify the rule when you configure the production.

To edit an existing rule definition, open it in the Rule Editor and select the  icon next to its name.

2.1.1 Exporting and Importing Rules

A rule definition, including its rule sets and rules, can be exported by navigating to **Interoperability > List > Business Rules**, and selecting **Export**. You can import a previously exported rule definition using the **Import** option.

Alternatively, you can also export and import rule classes from the **System Explorer > Globals** page of the Management Portal or via [your IDE](#).

2.2 About Rule Sets

When you create a new rule definition, a new rule set is created automatically. To define the rule set name and effective date range, or to create new rules sets, select the  icon next to the rule set name.

There are two types of rule sets:

- *General business rule sets*—A list of rules that are evaluated sequentially until one of them is found to be true. The rule that is found to be true determines the next action of the business process that invoked the rule. If none of the rules are true, the rule set returns a default value. You invoke this type of rule set using the BPL `<rule>` element.
- *Routing rule sets*—A rule set for use in message routing productions. Based on the types and contents of incoming messages (which you specify as constraints), the routing rule set determines the correct destination for each message and how to transform the message contents prior to transmission. You use a routing engine business process to invoke this type of rule set.

All rule sets have two properties:

- **Rule Set Name**—Identifier for the rule set.
- **Effective Range**—Defines the time during which the rule set is effective, that is, when its rules will be executed.

Typically, a rule definition includes only one rule set that is always in effect. However, a rule definition can include multiple rule sets as long as they are in effect at different times. Each time a business process invokes a rule, one and only one rule set is executed.

2.3 See Also

- [About Business Rules](#)
- [Working with Rules](#)
- [Expressions in Business Rules](#)
- [Debugging Routing Rules](#)

3

Working with Rules

This page briefly describes how to create [business rules](#) for [interoperability productions](#). A rule set contains one or more rules that you define to satisfy specific functions in a business process. Once you create a new rule definition in the Rule Editor (**Interoperability** > **Build** > **Business Rules**), you are ready to start adding rules to a rule set.

Though you can give each rule a name, it is not required. By default, InterSystems IRIS® names the rules in sequential order in the form rule#*n*. If you give the rule a user-defined name, it appears in the class definition and also appears in parentheses next to the internal rule name in the rule log. The value of *n* changes if you reorder the rules in a rule set.

3.1 Defining Constraints

If the rule set contains routing rules, you can define constraints such that when a message makes its way through the rule set, the rule logic is executed only if the message matches the defined constraints. Leaving a field blank will match all values. To set constraints for a rule, double-click the rule and define the following settings:

Source

Configuration name of *one* of the following items:

- A business service (for a routing interface)
- A message routing process (if another rule chains to this routing rule set)

Message Class

Identifies the production message object that is being routed by this rule. The value of this field depends on the routing rule type:

- *For a General Message Routing Rule*, you can click the ellipsis (...) next to the **Message Class** field to invoke the **Finder Dialog** and select the appropriate message class. You can choose the category of message class to narrow your choices.
- *For a Virtual Document Message Routing Rule*, you can choose from the list of defined virtual document classes.

Schema Category

For virtual document routing rules, identifies the category of the message class and specifies its structure. You can choose from the list of category types defined for your chosen virtual document class. The types may be built-in or imported from a custom schema.

Document Name

For virtual document routing rules, identifies the message structure. The acceptable values depend on the message class. You can choose from the list of category types defined for your chosen virtual document class. The types may be built-in or imported from a custom schema.

If you specify more than one value in the **Document Name** field, the rule matches any of the specified **Document Name** values and no others.

3.2 About If and Else Clauses

A rule can contain one or more **if** clauses and an **else** clause. Each clause can include actions such as **assign** or **return**.

The logic in an **if** clause can be executed only if the condition property associated with the clause holds true. The logic in an **else** clause can be executed only if *none* of the condition properties associated with the preceding **if** clauses holds true. When a rule contains multiple **if** clauses, only the logic in the first **if** clause where the condition property associated with the clause holds true is executed. See [Boolean Expressions](#).

As you develop rules, keep the following points in mind:

- Once the execution through a rule set encounters a **return** action, the execution of the rule set ends and returns to the business process that invoked the rule definition class.
- You can control the execution of more than one rule in a rule set by omitting the returns. In other words, if you want to check all rules, do not provide a **return** action within any of the rule clauses. You may then provide a value in a **return** action at the end of the rule set for the case where no rule clauses evaluate to true.
- Each **if** clause has a condition property. A common design for a general business rule set is one that contains one rule with a series of **if** conditions and returning a value depending on which condition is true. If you want to return a default value if none of the conditions is true, you can use the **else** clause with a return.
- A common design for a routing rule set is one that contains several rules each with a different constraint defined and each with one **if** clause describing how and where to route the message that matches the constraint.
- You can access property paths in virtual documents using the syntax described in [Virtual Property Path Basics](#).

3.3 About Actions

Each **if** or **else** clause in a rule can include actions, but they are not required. The actions in a clause are executed if and only if the condition associated with the clause holds true. The following actions are supported:

Rule Set Type	Action	Description
All	assign	Assigns values to properties in the business process execution context.
All	return	Returns to the business process without further execution of the rule. For general rules it also returns the indicated value to the result location.
All	trace	Adds the information you enter into the Event Log when this specific part of the rule is executed. For details, see <trace> .
All	debug	Adds the expression text and value to the Rule Log when this specific part of the rule is executed. The debug action is executed only if the router business process RuleLogging property specifies the d flag, For details on the RuleLogging property, see Rule Logging .
Segmented Virtual Document Routing Rule or HL7 Message Routing Rule	foreach	Loops through a repeating segment. A segment may repeat if it is designated as a repeating segment, is in a repeating loop, or both. See Using the foreach Action for more details.
Routing Rule	send	When evaluated by a routing engine business process, this action sends the message to a particular target after optionally transforming it. For the ability to pass data to the data transformation, see Passing Data to a Data Transformation .
Routing Rule	delete	When evaluated by a routing engine business process, this action deletes the current message.
Routing Rule	delegate	When evaluated by a routing engine business process, this action delegates the message to a different rule.

The **send**, **delete**, and **delegate** actions should not be used within a BPL `<rule>`. If you include them, the action will not be executed and instead a string value will be returned that includes the given action.

You must ensure that you construct rule sets such that they are logically sound and result in the rule set being executed as you intended. For example, while it might make sense to set a default return value if none of the rules in a rule set are executed, it does not make sense to do so if you have created the rule set such that one rule is always executed. Typically, most actions reside in the **if** clauses of rules.

3.3.1 Using the foreach Action

You can use the **foreach** action within an **if** or an **else**. The **foreach** action allows you to loop through a repeating segment and reference any of the fields within the segment.

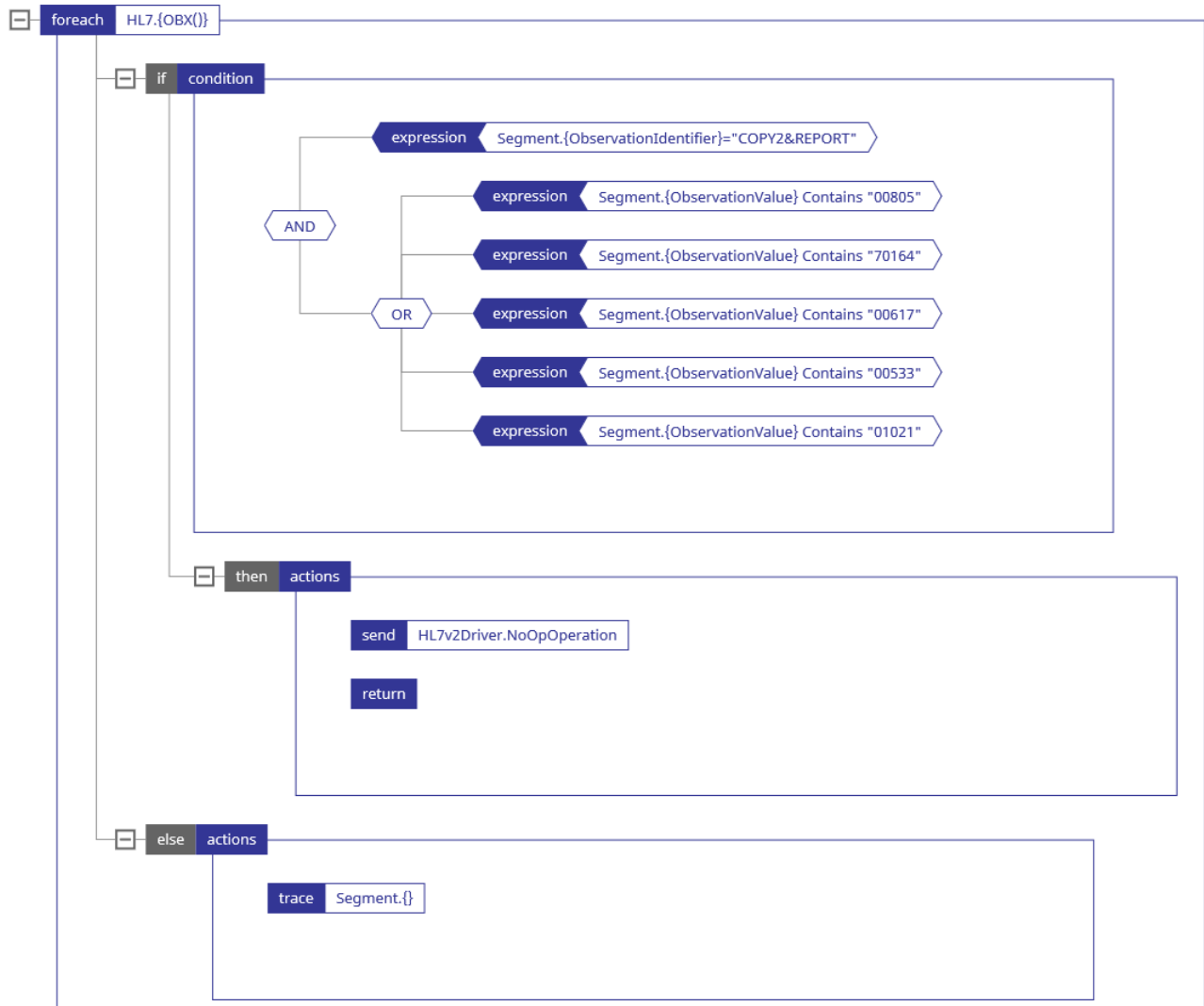
You specify the repeating segment in the **propertypath** property of the **foreach** action using the syntax described in [Virtual Property Path Basics](#). For example, to access the OBX segments in the repeating OBXgrp of an HL7 document, you can specify `HL7 . {OBXgrp ( ) . OBX}`, where the empty parentheses indicate the repeating group. A **foreach** action can contain one or more **if** clauses and an **else** clause. Within the clauses, you specify actions to execute when the conditions in the clauses hold true.

For example, you can use a **foreach** action to determine when a field in a repeating segment contains a particular value, and then specify a **send** action to route a message when the value is present. To reference the specific field, you can use `Segment.{<field-name>}` — for example: `Segment.{ObservationIdentifier}`.

The **if** and **else** clauses in a **foreach** action can contain one or more **rule** nodes. However, you cannot nest **foreach** actions.

When the rule executes a **return** action within a **foreach** loop, it exits the entire rule set, not just the loop or rule.

The following extract shows the use of a **foreach** action within a business rule. The action iterates through a repeating **OBX** segment in an HL7 document to determine when the **ObservationIdentifier** field contains certain string values. When the values are found, the rule sends the document to a file operation. When the values are not found, the rule logs an entry in the Event Log using a trace action.



For reasons of space, this diagram does not show the **if** that contains this **foreach**.

3.4 Disabling a Rule

If you would like to prevent a rule set from executing a rule, but do not want to delete the rule, you can disable it. Simply double-click the rule, and select **Disable**.

3.5 Passing Data to a Data Transformation

A Send action can invoke a data transformation before sending the message to a target within the production. This data transformation can use its `aux` variable to obtain information from the rule. Some of this data, for example the name of the rule and the reason the rule was fired, is available to the transformation without making changes to the rule class.

In order to pass additional information to the transformation, you need to edit the rule class in an IDE to assign values to properties of the class. A value assigned to the `RuleUserData` property of the rule class is available to the transformation if it accesses the `aux.RuleUserData` variable. A value assigned to the `RuleActionUserData` property of the rule class is available to the transformation as `aux.RuleActionUserData`.

For additional information about accessing the `aux` variable in a transformation, see the list of valid expressions in [DTL Syntax Rules](#).

3.6 See Also

- [About Business Rules](#)
- [Getting Started](#)
- [Expressions in Business Rules](#)
- [Debugging Routing Rules](#)

4

Expressions in Business Rules

This page provides information on the variables, operators, functions, and expressions you can use in a [business rule](#).

4.1 The context Variable

A business rule can refer to the *context* variable defined by the BPL that calls the business rule.

The *context* variable contains any data that needs to be persisted during the life cycle of the business process. This variable is an object with properties and defined when creating the BPL business process; see [Defining the context Object](#). To refer to a property of the context object, use dot syntax and the property name, as in: *context.MyData*

Note: Property names are case-sensitive and must *not* be enclosed in quotes, for example, PlaceOfBirth.

For context properties that contain collections such as lists and arrays, InterSystems IRIS supports several retrieval methods from within business rules, including **Count()**, **Find()**, **GetAt()**, **GetNext()**, **GetPrevious()**, **IsDefined()**, **Next()**, and **Previous()**. For more information, see Working with Collections.

4.2 The Document Variable

A business rule can also refer to the *Document* variable, which represents the message object. The Document variable is available only if you have a [constraint](#) set with the **Message Class** property. Setting the **Message Class** enables the Routing Rule UI to offer up suggested properties.

4.3 Available Operators

When defining an expression, you can select one of the following arithmetic operators:

Operator	Meaning
+	Plus (binary and unary)
–	Minus (binary and unary)
*	Times
/	Divide

Additionally, the following logical operators are supported and return an integer value of 1 (true) or 0 (false):

Operator	Meaning	Expression is true when...
AND (&&)	And	Both values are true.
OR ()	Or	At least one of the values is true. Both values may be true, or only one true.
!	Not (unary)	The value is false.
=	Equals	The two values are equal.
!=	Does not equal	The two values are <i>not</i> equal.
>	Is greater than	The value to the left of the operator is <i>greater</i> than the value to the right of the operator.
<	Is less than	The value to the left is <i>less</i> than the value to the right.
>=	Is greater than or equal to	The value to the left is greater than the value to the right, <i>or</i> if the two values are equal.
<=	Is less than or equal to	The value to the left is less than the value to the right, <i>or</i> if the two values are equal.
[	Contains	The string contains the substring to the right. Pattern matching for Contains is exact. If the value at left is “Hollywood, California” and the value at right is “od, Ca”, there is a match, but a value of “Wood” does not match.

Lastly, you can use the following string operators:

Operator	Meaning
&	Concatenation operator for strings.
_	Binary concatenation to combine string literals, expressions, and variables.

When more than one operator is found in an expression, the operators are evaluated in the following order of precedence, from first to last:

1. Any of the following logical operators: ! = != < > <= >= [
2. Multiplication and division: * /
3. Addition and subtraction: + –
4. String concatenation: & _
5. Logical AND: &&
6. Logical OR: ||

4.4 Available Functions

Within a rule definition, an expression can include a call to one of the InterSystems IRIS [utility functions](#). These include mathematical or string processing functions similar to those that exist in other programming languages. When defining the expression, simply select a function from the drop-down list.

See [Utility Functions for Use in Productions](#).

4.5 Expression Examples

Within a rule definition, an *expression* is a formula for combining values and properties to return a value. The following table includes examples of expressions along with their computed values:

Expression	Computed value
$((2+2)*5)/154.3$	0.129617628
"hello" & "world"	"helloworld"
Age * 4	If Age is a <i>context</i> property (a property in the general-purpose, persistent <i>context</i> variable, which you can define using the <context> and <property> elements in BPL) and has the numeric value 30, the value of this expression is 120.
$1+2.5*2$	6
$2*5$	10
Min(Age, 80, Limit)	This expression uses the built-in Min() function . If Age is a <i>context</i> property with the value 30 and Limit (likewise a property) has the value 65, the value of this expression is 30.
Round(1/3, 2)	This expression uses the built-in Round() function . The result is 0.33.
$x < 65 \&\& A = "F" \mid x > 80$	This expression uses the operator precedence conventions that are described in Expression Operators . If A is a <i>context</i> property with the string value F, and x (likewise a property) has the integer value 38, this expression has the integer value 1. In InterSystems IRIS, an integer value of 1 is true and an integer value of 0 means false.
Min(10, Max(X, Y))	This expression uses the Min() and Max() functions . If X is a <i>context</i> property with the numeric value 9.125, and Y (likewise a property) has the numeric value 6.875, the value of this expression is 9.125.
$((x=1) \mid \mid (x=3)) \&\& (y=2)$	This expression uses parentheses to clarify precedence in a complex logical relationship.

When you select a property that takes an expression as its value, a blank text field appears at the top of the rule set diagram. You must ensure that you use the appropriate syntax for the property since the text field enables you to specify any string. Consider the following rules when you formulate an expression:

- An expression can include the values described in previous sections: numbers, strings, *context* properties, other expressions, [functions](#), or any valid combination of these.
- White spaces in expressions are ignored.
- You can use any of the supported operators in an expression.

- If you want to override the default operator precedence, or if you want to make an expression easier to read, you can use parentheses to group parts of the expression and indicate precedence. For example, consider the following expression, which results in a value of 6:

$1+2.5*2$

If you change the expression as follows, the result becomes 7:

$(1+2.5)*2$

- Business rules support parentheses to group complex logical expressions such as $((x=1) \ || \ (x=3)) \ \&\& \ (y=2))$.

4.6 Boolean Expressions

In a rule definition, a *condition* consists of two values and a comparison operator between the values, for example:

```
Amount <= 5000
```

If a **condition** is not **true**, it is **false**. There are no other possible values for the **condition** property. A result that may be only true or false is called a *boolean* result. InterSystems IRIS stores boolean results as integer values, where 1 is true and 0 is false. In most cases, you do not need to use this internal representation. However, for a routing rule, you may want to execute the **if** clause that corresponds to the **condition** property any time the **constraint** for the rule holds true. In this case, you can set the **condition** property to 1.

A **condition** property can contain more than one condition. InterSystems IRIS evaluates and compares all the conditions in the property before determining whether to execute the corresponding rule. The logic between conditions is determined by AND or OR operators. For example, consider a **condition** property with the following value:

```
IF Amount <= 5000
AND CreditRating > 5
OR CurrentCustomer = 1
```

The same value appears in the **Rule Editor** as follows



The value contains three conditions: $\text{Amount} \leq 5000$, $\text{CreditRating} > 5$, $\text{CurrentCustomer} = 1$. Each condition could be true or false. InterSystems IRIS evaluates the conditions individually before evaluating the relationships between them defined by the AND and OR operators.

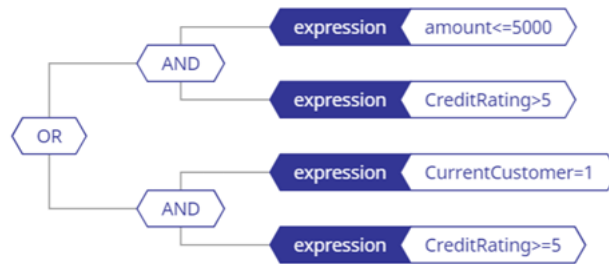
The AND and OR operators can operate only on true and false values. That is, the operators must be positioned between two boolean values and return a single boolean result as follows:

Operator	Result is true when...
AND	Both values are true.
OR	At least one of the values is true, or both are true. If one of the values is false and the other is true, then the result (as a whole) is still true.

If a **condition** property contains multiple AND or OR operators, the AND operators take precedence over the OR operators. Specifically, all the AND operations are performed first. Then, the OR operations are performed. For example, consider the following set of conditions:

```
IF Amount <= 5000
AND CreditRating > 5
OR CurrentCustomer = 1
AND CreditRating >= 5
```

The same set of conditions appears in the **Rule Editor** as follows:



InterSystems IRIS evaluates the conditions as follows:

```
IF (Amount <= 5000 AND CreditRating > 5)
OR (CurrentCustomer = 1 AND CreditRating >= 5)
```

That is, the whole set of conditions is true if either or both of the following statements is true:

- Someone requests an amount less than 5,000 *and* has a credit rating better than average.
- A current bank customer requests any amount *and* has a credit rating greater than or equal to the average.

If both statements are false, then the set of conditions (as a whole) is false.

To explain another way, InterSystems IRIS evaluates the set of conditions by taking the following steps:

1. Determine whether the result of the following AND expression is true or false:

```
IF Amount <= 5000
AND CreditRating > 5
```

Suppose this result is called “SafeBet.”

2. Determine whether the result of the following AND expression is true or false:

```
IF CurrentCustomer = 1
AND CreditRating >= 5
```

Suppose this result is called “KnownEntity.”

3. Determine whether the result of the following OR expression is true or false:

```
IF SafeBet is true
OR KnownEntity is true
```

If SafeBet is true and KnownEntity is false, then the set of conditions is true. Similarly, if SafeBet is false and KnownEntity is true, then the set of conditions is true. Lastly, if both SafeBet and KnownEntity are true, then the set of conditions is true.

5

Debugging Routing Rules

This page describes how to test routing [rules](#) without sending the message through the entire [interoperability production](#). It also contains flow diagrams that can help you debug problems in routing rules defined for EDI messages in a production.

5.1 Testing Routing Rules

Using the **Test** button of the Rule Editor, you can see whether a message triggers any of the routing rules without having to send the message through the entire production. Running this test does not transform or send the message, but any functions in the condition are executed as if the message ran through the production.

If you want to test a rule's constraint that is based on the source of the message, use the **Production Source** field to specify the business host in the production that is sending the message. You can use the drop down menu to choose the business host from a list.

You can use the **Context** field to specify the contents of the message in one of three ways:

- Specify User Input and then click next to paste the [raw text of a message](#).
- Specify the Document Body ID of an existing message. You can find the Document Body ID for a message by looking at the **<Object Id>** field on the **Body** tab of the Message Viewer.
- Specify the Message Header ID of an existing message. If the **Production Source** field is blank, the Source Config Name in the message header is used as the source. You can find the Message Header ID for a message by looking at the **<Object Id>** field on the **Header** tab of the Message Viewer.

5.1.1 Testing with Raw Text of a Message

For a virtual document message routing rule, you can test the rule with the raw text of a message. To do so:

1. Optionally, in the **Production Source** field, enter the business host that is sending the message.
2. Choose **User Input** from the **Context** drop-down list.
3. Click the **Next** button.
4. Paste the raw text of the message in the **Content** text field.

This can be the text of an HL7 message or an X12 message.

5. Enter any other constraints that you want to test by entering information in the **DocType** field or by selecting from the **Category** or **Name** drop downs.

6. Click **Submit**.

5.1.2 Test Results

The test results will tell you whether the message met a rule's constraint and whether an if or else clause was triggered.

5.1.3 Security Requirements

A user must have the correct security privileges to test routing rules. They must have USE permissions for the **%Ens_RuleLog** and **%Ens_TestingService** resources. In addition, they must have Select SQL privileges on **Ens_Rule.log** and **Ens_Rule.DebugLog** tables.

5.2 Strategies for Debugging Routing Rules

This section describes strategies for debugging the routing rules in an EDI message routing production.

The primary symptom for problems in routing rules is that the message does not reach its destination. Perhaps the message reaches a point along the way, such as a business operation or routing process within the routing production, but it does not reach its target destination, which is typically an application server external to InterSystems IRIS®.

Follow the problem-solving sequence captured in the five decision trees below, starting with the initial triage in decision tree [A](#).

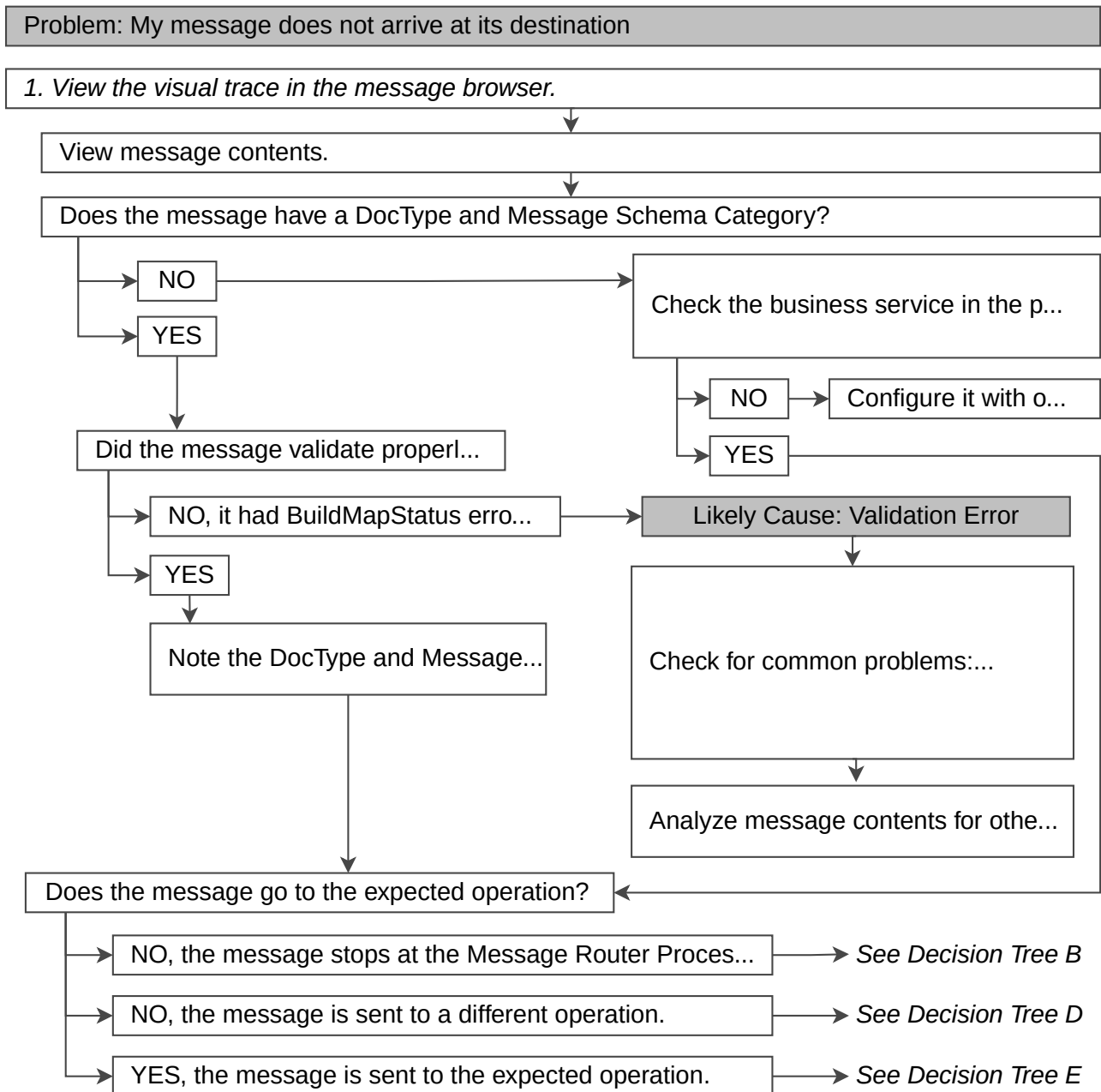
Figure 5-1: Solving Problems with Routing Rules, Initial Triage — Decision Tree A

Figure 5-2: Solving Problems with Routing Rules — Decision Tree B

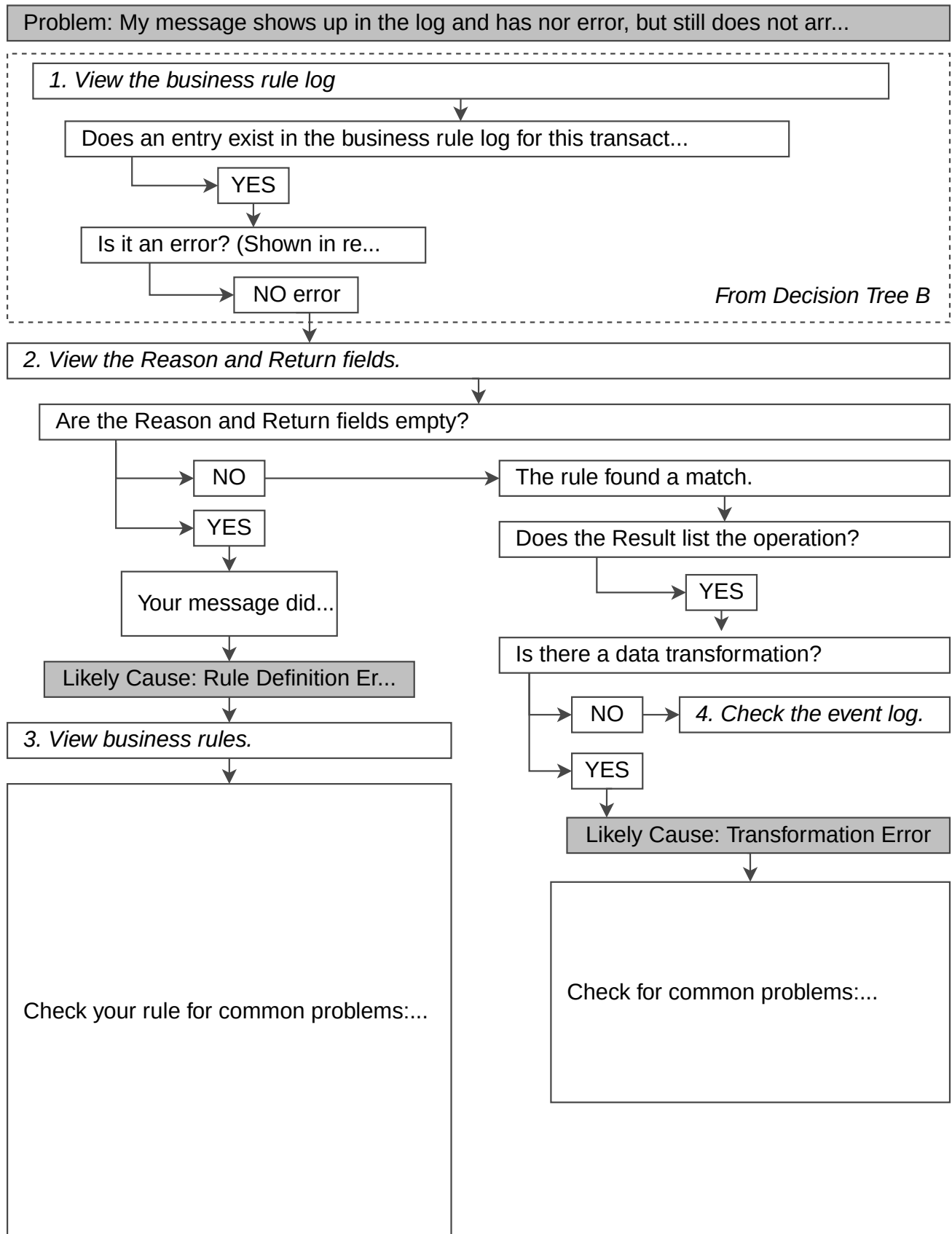
Figure 5-3: Solving Problems with Routing Rules — Decision Tree C

Figure 5-4: Solving Problems with Routing Rules — Decision Tree D

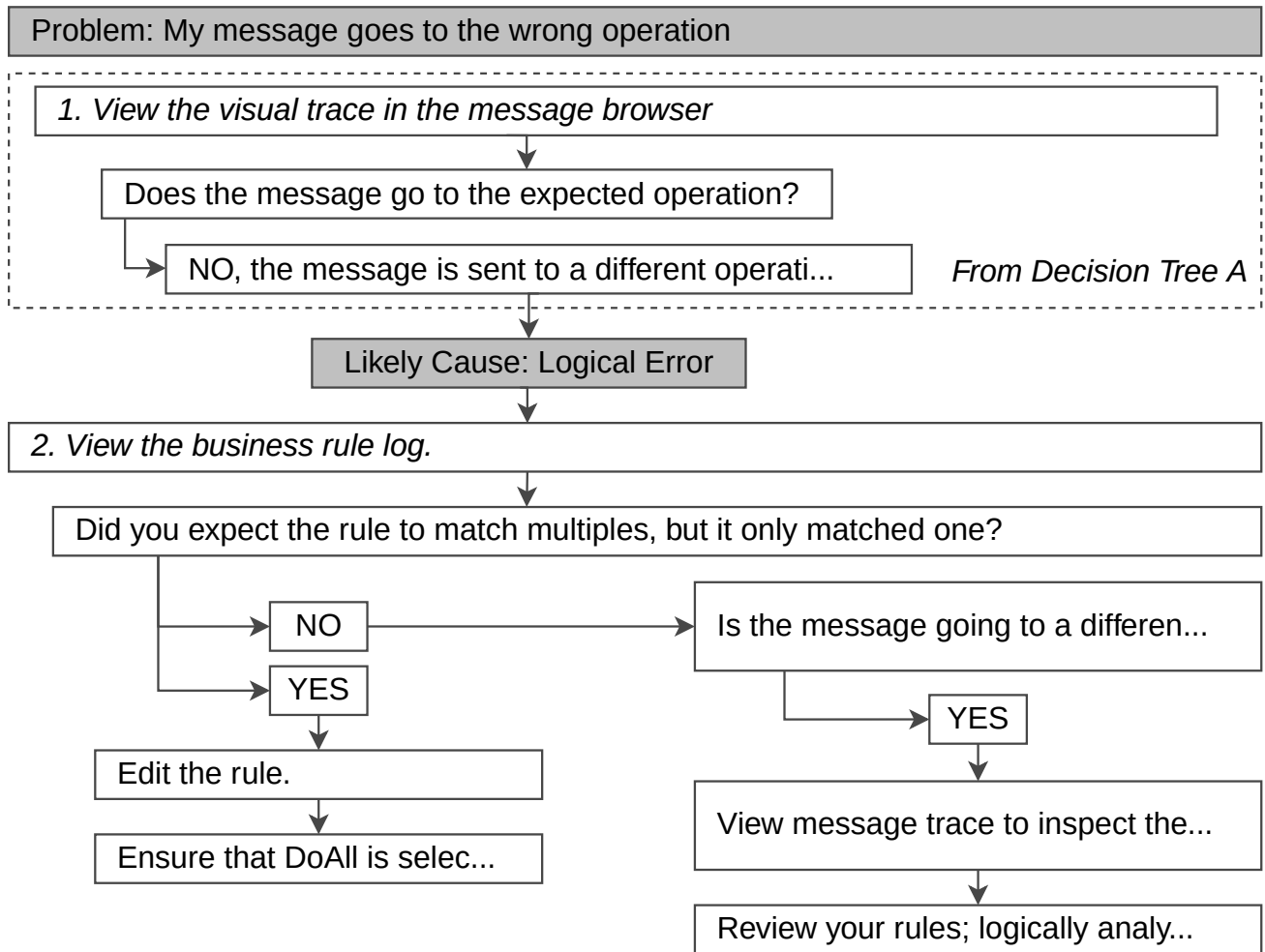
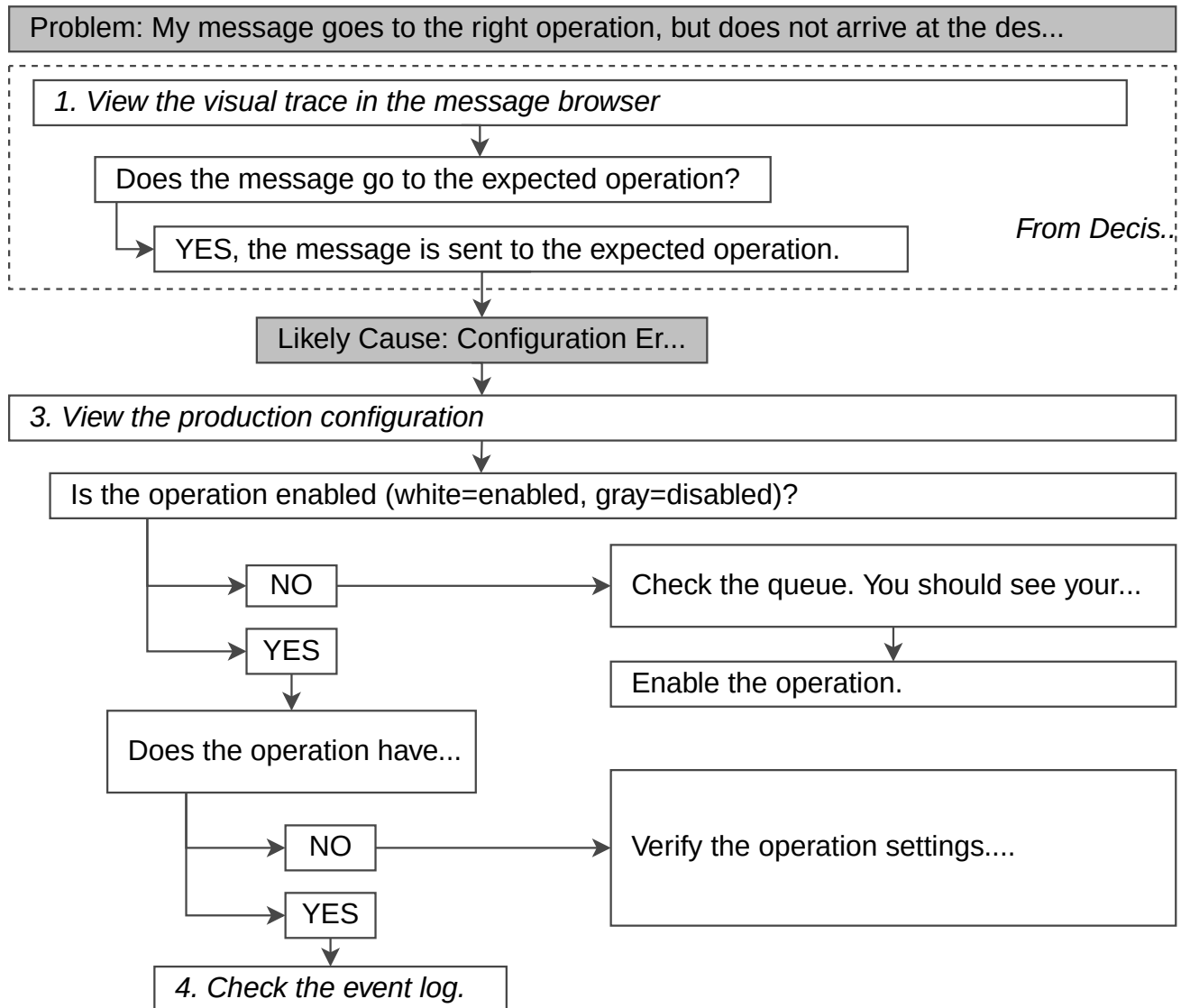


Figure 5-5: Solving Problems with Routing Rules — Decision Tree E



5.3 See Also

- [About Business Rules](#)
- [Getting Started](#)
- [Expressions in Business Rules](#)
- [Working with Rules](#)

A

Utility Functions for Use in Productions

This page describes the *utility functions* that you can use in [business rules](#) and [DTL data transformations](#) for [interoperability productions](#). These include mathematical or string processing functions such as you may be accustomed to using in other programming languages.

To define your own functions, see [Defining Custom Utility Functions](#).

A.1 Built-in Functions

The following lists the utility functions built into InterSystems IRIS.

Note: For boolean values, 1 indicates true and 0 indicates false.

Contains(*value*,*string*)

Returns 1 (true) if *value* contains the substring *string*; otherwise 0 (false). The following example tests for ABC within the value of a property of the source message:

```
Contains(source.SampleProperty, "ABC")
```

ConvertDateTime (*value*,*in*,*out*,*file*)

Reads the input string *value* as a time stamp in *in* format, and returns the same value converted to a time stamp in *out* format. See [Time Stamp Specifications for Filenames](#).

The default for *in* and *out* is %Q. Any %f elements in the *out* argument are replaced with the *file* string. If *val* does not match the *in* format, *out* is ignored and *val* is returned unchanged.

CurrentDateTime(*format*)

Returns a string representing a date/time value in the given format. For a list of possible formats, see the Date and Time Expansion section of the class reference for the **FormatDateTime** method. For example, `CurrentDateTime("%H")` returns the current hour in 24-hour format as a 2-digit number. The default format is ODBC format (%Q) in the server's local timezone.

DoesNotContain(*value*,*string*)

Returns 1 (true) if *value* does not contain the substring *string*. For example:

```
DoesNotContain(source.SampleProperty, "ABC")
```

DoesNotIntersectList(*value*,*items*,*srcsep*,*targetsep*)

Returns 1 (true) if no item in the given source list (*value*) appears in the target list (*items*). For details on the arguments, see **IntersectsList()**.

DoesNotMatch(*value*,*pattern*)

Returns 1 (true) if *value* does not match the pattern specified by *pattern*. *pattern* must be a string that uses syntax suitable for the ObjectScript pattern match operator. For example:

```
DoesNotMatch(source.SampleProperty, source, "ssn?3N1"" - "2N1"" - ""4N")
```

DoesNotStartWith(*value*,*string*)

Returns 1 (true) if *value* does not start with the substring *string*. For example:

```
DoesNotStartWith(source.SampleProperty, "ABC")
```

Exists(*tablename*,*value*)

The **Exists()** function provides a way to predict the results of the **Lookup()** function. **Exists()** returns 1 (true) if *value* is a key defined within the table identified by *tablename*; otherwise it returns 0 (false).

The *tablename* value must be enclosed in double quotes, for example:

```
Exists("Alert", "Priority_FileOperation")
```

You can omit the double quotes around *value*, if that argument is a number.

If(*value*,*true*,*false*)

If the argument *value* evaluates to 1 (true), the this function returns the string value of its *true* argument; otherwise it returns the string value of its *false* argument.

In(*value*,*items*)

Returns 1 (true) if *value* is found in the comma-delimited string *items*.

You can use this function with a list that uses a custom separator (rather than a comma). To do so, append two commas to the *items* list, followed by the custom separator. The system will parse the *items* argument, find the two sequential commas and then interpret the following character as the separator to use. For example (here using the equivalent method call from **Ens.Util.FunctionSet**):

```
ENSLIB>set items="a|b|c,,|"
```

```
ENSLIB>write ##class(Ens.Util.FunctionSet).In("a", items)  
1
```

You can also use this function with a list in which each item is wrapped by a prefix and a suffix character. To do so, append two commas to the *items* list, followed by the prefix character and then the suffix character. For example:

```
ENSLIB>set items="<a><b><c>,,<>"
```

```
ENSLIB>w ##class(Ens.Util.FunctionSet).In("a", items)  
1
```

InFile(*value*,*filename*)

Returns 1 (true) if *value* is found in the file whose name is *filename*.

InFileColumn(...)

The function **InFileColumn()** can have as many as 8 arguments. The full function signature is:

InFileColumn(value, file, columnId, rowSeparator, columnSeparator, columnWidth, lineComment, stripPadChars)

InFileColumn() returns 1 (true) if *value* is in the specified column in a table-formatted text *file*. Arguments are as follows:

- *val* (required) is the value.
- *file* (required) is the text file.
- Default *columnId* is 1.
- Default *rowSeparator* is ASCII 10. A negative *rowSeparator* value indicates the row length.
- Default *columnSeparator* is ASCII 9. If *columnSeparator* is 0, the format of the file is said to be “positional.” In this case *columnId* means character position and *columnWidth* means character count.
- Default *columnWidth* is 0.
- Default *lineComment* is an empty string.
- Default *stripPadChars* consists of a blank space followed by ASCII 9.

IntersectsList(value,items,srcsep,targetsep)

Returns 1 (true) if any item in the given source list (*value*) appears in the target list (*items*). The arguments *srcsep* and *targetsep* specify the list separators in the source and target lists respectively; for each of these, the default is "><", which means that the lists are assumed to have the form "<item1><item2><item3>".

The **IntersectsList()** function works well with the [square bracket \[\] syntax](#) to match values of a virtual document property. If there is more than one instance of the segment type in a message, the square bracket syntax returns the multiple values in a string like <ValueA><ValueB><ValueC>.

If the target list has only a single item, this function is essentially the same as the **Contains()** function. If the source list has only a single item, this function is essentially the same as the **In** function.

Length(string,delimiter)

Returns the length of the given string. If you specify *delimiter*, this function returns the number of substrings based on this delimiter. For example, to get the number of characters in a property, do this:

```
Length(source.SampleProperty)
```

If the property contains commas, and you want to find how many comma-separated pieces it has, do this:

```
Length(source.SampleProperty, ", ")
```

Like(string,pattern)

Returns 1 (true) if the given value (*string*) satisfies a SQL Like comparison with the given pattern string (*pattern*). In SQL Like patterns, % matches 0 or more characters, and _ matches any single character. Note that an escape character can be specified by appending "%%" to the pattern, e.g. "%SYSVAR_#_%%%" to match any value string that starts with "%SYSVAR" followed by any single character, an underscore, and anything else.

Lookup(table,keyvalue,default, defaultOnEmptyInput)

The **Lookup()** function searches for the key value specified by *keyvalue* in the table specified by *table* and returns its associated value. This returned value is equivalent to the following global:

```
^Ens.LookupTable(table,keyvalue)
```

The *table* value must be enclosed in double quotes, for example:

```
Lookup("Gender", source.{PID:Sex}, , "U")
```

If the key is not found in the table, the **Lookup()** function returns the default value specified by the *default* parameter. The *default* parameter is optional, so if it is not specified and **Lookup()** does not find a matching key, it returns an empty string. An exception is that if either the key value or the lookup table is empty, the **Lookup()** function returns either the default value or the empty string depending on the value of the *defaultOnEmptyInput* parameter as follows:

defaultOnEmptyInputValue	key value and lookup table	Lookup() returns
0	either key value or lookup table is empty	empty string
1	key value is empty	empty string
1	lookup table is empty but key value is not	default value
2	lookup table is empty	empty string
2	key value is empty but lookup table is not	default value
3	either key value or lookup table is empty	default value

The default value of the *defaultOnEmptyInput* parameter is 0.

The **Exists()** function returns true if a **Lookup()** function with the same parameters would find the key value in the lookup table.

Matches(value,pattern)

Returns 1 (true) if *value* matches the pattern specified by *pattern*. *pattern* must be a string that uses syntax suitable for the ObjectScript pattern match operator. Any double quotation marks in this string must be double quoted. For example, to determine whether the value of a property matches the form of a valid U.S. Social Security Number, do this:

```
Matches(source.SampleProperty, source, "ssn?3N1"" - "2N1"" - ""4N")
```

Max(...)

Returns the largest of a list of up to 8 values. List entries are separated by commas. For example:

```
Max(source.SampleProperty1, source.SampleProperty2, source.SampleProperty3, 10000)
```

This returns the value of the largest property (of those listed), or 10000, whichever is greater.

Min(...)

Returns the smallest of a list of up to 8 values. List entries are separated by commas. For example:

```
Min(source.SampleProperty1, source.SampleProperty2, source.SampleProperty3, 0)
```

This returns the value of the smallest property (of those listed), or 0, whichever is smaller.

Not(*value*)

Returns 0 (false) if *value* is 1 (true); 1 (true) if *value* is 0 (false).

NotIn(*value,items*)

Returns 1 (true) if *value* is not found in the comma-delimited string *items*.

NotInFile(*value,filename*)

Returns 1 (true) if *value* is not found in the file whose name is *filename*.

NotLike(*string,pattern*)

Returns 1 (true) if the given value (*string*) does not satisfy a SQL Like comparison with the given pattern string (*pattern*). See **Like()**.

Pad(*value,width,char*)

Reads the input string *value*. Adds enough instances of *char* to widen the string to *width* characters. If *width* is a positive value, the padding is appended to the right-hand side of the *value* string. If *width* is a negative value, the padding is prepended to the left-hand side of the *value* string.

Piece(*value,char,from,to*)

If the delimiter character *char* is present in the string *value*, this separates the string into pieces. If there are multiple pieces in the string, *from* and *to* specify which range of these pieces to return, starting at 1. If multiple pieces are returned, the delimiter in the return string is the same as the delimiter in the input string.

If you omit the delimiter character *char* and the *from* and *to* arguments, this function assumes that *value* is a comma-separated list and returns the first item from that list (A, in this example):

```
Piece("A,B,C,D,E,F")
```

For the *to* argument, you can use "*" to refer to the last position in list. For both *from* and *to* arguments, you can use syntax that refers to positions relative to the last position. For example "* - 1" means the position just before the last position.

The default *char* is a comma, the default *from* is 1, and the default *to* is *from* (return one piece). For details, see the ObjectScript \$PIECE function. Note that in contrast to the ObjectScript function, you must use double quotes around "*" syntax.

ReplaceStr(*value,find,repl*)

Starting with the input string *value*, replaces any occurrences of string *find* with the string *repl*, and returns the resulting string. For example:

```
ReplaceStr(source.SampleProperty,"abc","ABC")
```

This example finds any occurrence of abc in the given property and replaces that with ABC.

Note: Use **ReplaceStr()** instead of the **Replace()** function, which has been deprecated.

See also **Translate()**.

RegexMatch(*string,regex*)

Given an input string *string* and a regular expression *regex*, returns 1 if *string* matches the regular expression; returns 0 otherwise.

Round(value,n)

Returns *value* rounded off to *n* digits after the decimal point. If *n* is not provided (that is, *Round(value)*) the function drops the fractional portion of the number and rounds it to the decimal point, producing an integer. For example:

```
Round(source.SampleNumericProperty, 2)
```

Rule(rulename,context,activity)

Evaluates the rule specified in the *rulename* with the given *context* object and the given *activity* label for the Rule Log and returns the value.

Schedule(ScheduleSpec, ODBCDateTime)

Evaluates the state of the given *ScheduleSpec* string, named Schedule or Rule at the moment given by *ODBCDateTime*. If *ScheduleSpec* begins with '@' it is a Schedule name or Rule name, otherwise a raw Schedule string. If *ODBCDateTime* is blank, the evaluation is done for the current time.

StartsWith(value,string)

Returns 1 (true) if *value* starts with the substring *string*; otherwise 0 (false). The following example tests whether a property of the source message starts with ABC:

```
StartsWith(source.SampleProperty, "ABC")
```

Strip(value,act,rem,keep)

Reads the input string *value*. Removes any characters matching the categories specified in the *act* template and the *rem* string, while retaining any characters found in the *keep* string. Returns the resulting string. For details on these arguments, see \$ZSTRIP.

For example, to remove numeric characters from a property of the source message:

```
Strip(source.SampleProperty, "*N")
```

To remove leading and trailing whitespace:

```
Strip(source.SampleProperty, "<>W")
```

SubString(string,n,m)

Returns a substring of a string *string*, starting at numeric position *n* and continuing until numeric position *m*. The number 1 indicates the first character in the string. If *m* is not provided (that is, *SubString(string,n)*) the function returns the substring from position *n* to the end of the string.

For example, to get the first five characters of a property, do this:

```
SubString(source.SampleProperty, 1, 5)
```

To get characters 10–15 of a property, do this:

```
SubString(source.SampleProperty, 10, 15)
```

To get characters starting at 15 and going to the end, do this:

```
SubString(source.SampleProperty, 15)
```

ToLower(*string*)

Returns the string *string* converted to lowercase. For example:

```
ToLower(source.SampleProperty)
```

ToUpper(*string*)

Returns the string *string* converted to uppercase. For example:

```
ToLower(source.SampleProperty)
```

Translate(*value,in,out*)

Reads the input string *value*. Translates each occurrence of a character in string *in* to the character at the corresponding position in string *out*, and returns the resulting string.

```
Translate(source.SampleProperty, "abc", "ABC")
```

This example finds any occurrence of a in the given property and replaces that with A, finds any occurrence of b and replaces that with B, and finds any occurrence of c and replaces that with C.

See also **ReplaceStr()**.

Note: These functions are defined by methods in the class `Ens.Util.FunctionSet`.

A.2 Usage Differences between Business Rules and DTL

The syntax for calling a utility function is different for business rules and for DTL:

- In a business rule, simply refer to the utility function by name, along with any arguments:

```
ToUpper(value)
```

- In DTL, use two leading dots before the function name, along with any arguments:

```
..ToUpper(value)
```

The Business Rule Editor and the DTL Editor handle this automatically.

A.3 See Also

- [Expressions in Business Rules](#)
- [Defining Custom Utility Functions](#)

